Documentación proyecto reproductor musical – “Blend”

Para la realización de esta práctica se ha requerido la utilización de la herramienta Howler.js, para la gestión del audio.

La gestión del sonido del reproductor a través de etiquetas básicas de HTML5 (<audio>) puede traer inconvenientes, por ello hemos incorporado esta potente librería de JavaScript llamada Howler.js.

¿Por qué lo utilizamos?

Controlar el audio nativo en los navegadores puede generar inconsistencias o ser limitado. Con Howler.js evitamos problemas de compatiblidad entre navegadores (Chrome,Safari,Edge,Firefox, etc.), además de que podemos controlar fácilmente la reproducción (pausa,volúmen, tiempo exacto) y lo más importante, nos otorga una vía de acceso directo a la **Web Audio API** del navegador, la cual necesitamos para construir el ecualizador y visulizador.

¿Cómo vinculamos Howler.js a nuestro proyecto?

Hemos optado vincularlo en el propio archivo index.html, dentro de la etiqueta <head>, antes de cargar el propio archivo scripts.js que gestiona todo el reproductor:

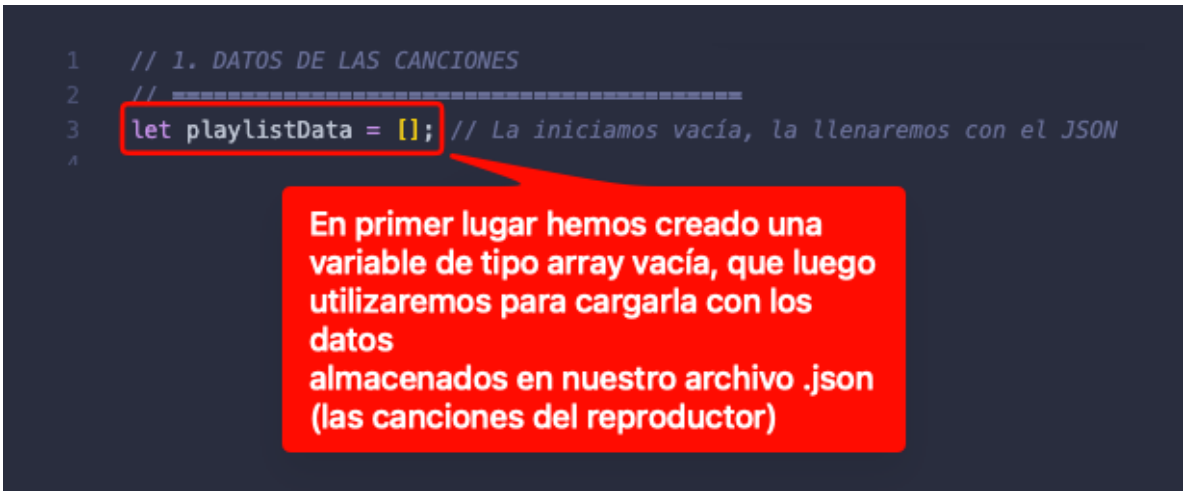
```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Blend - Inicio</title>
    <link rel="stylesheet" href="styles.css">
    <script src="https://cdnjs.cloudflare.com/ajax/libs/howler/2.2.4/howler.min.js"></script>
    <script src="scripts.js" defer></script>
```

Esta metodología implica utlizar una red de distribución de contenidos (CDN). Consiste en que nuestro proyecto no necesita descargar físicamente el código fuente de la librería y guardarlo entre las carpetas de nuestro reproductor, sino que el navegador del usuario lo obtiene de internet al cargar la página.

Como lo colocamos ANTES de nuestro archivo script, nos aseguramos de que cuando nuestro código empieze a leerse, las funciones de Howler ya existan en la memoria.

DESGLOSE DE NUESTRO MOTOR – SCRIPTS.JS

1) Creación del array que almacenará los datos recibidos del JSON.



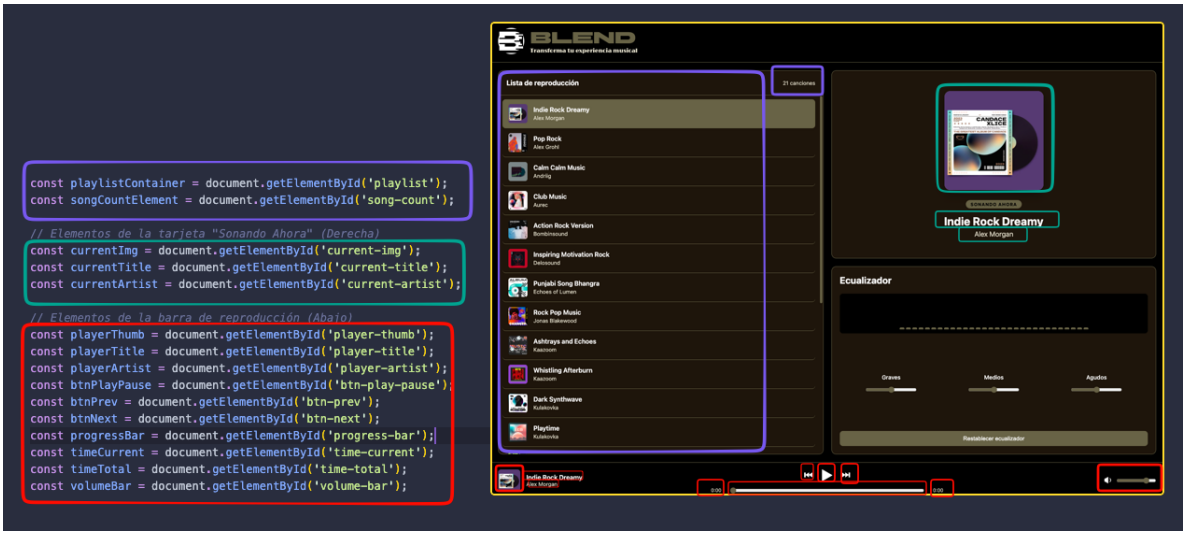
Como podemos ver, en la raiz de nuestro proyecto tenemos el archivo playlist.json que almacena la información sobre las canciones que formaran parte de nuestro reproductor:



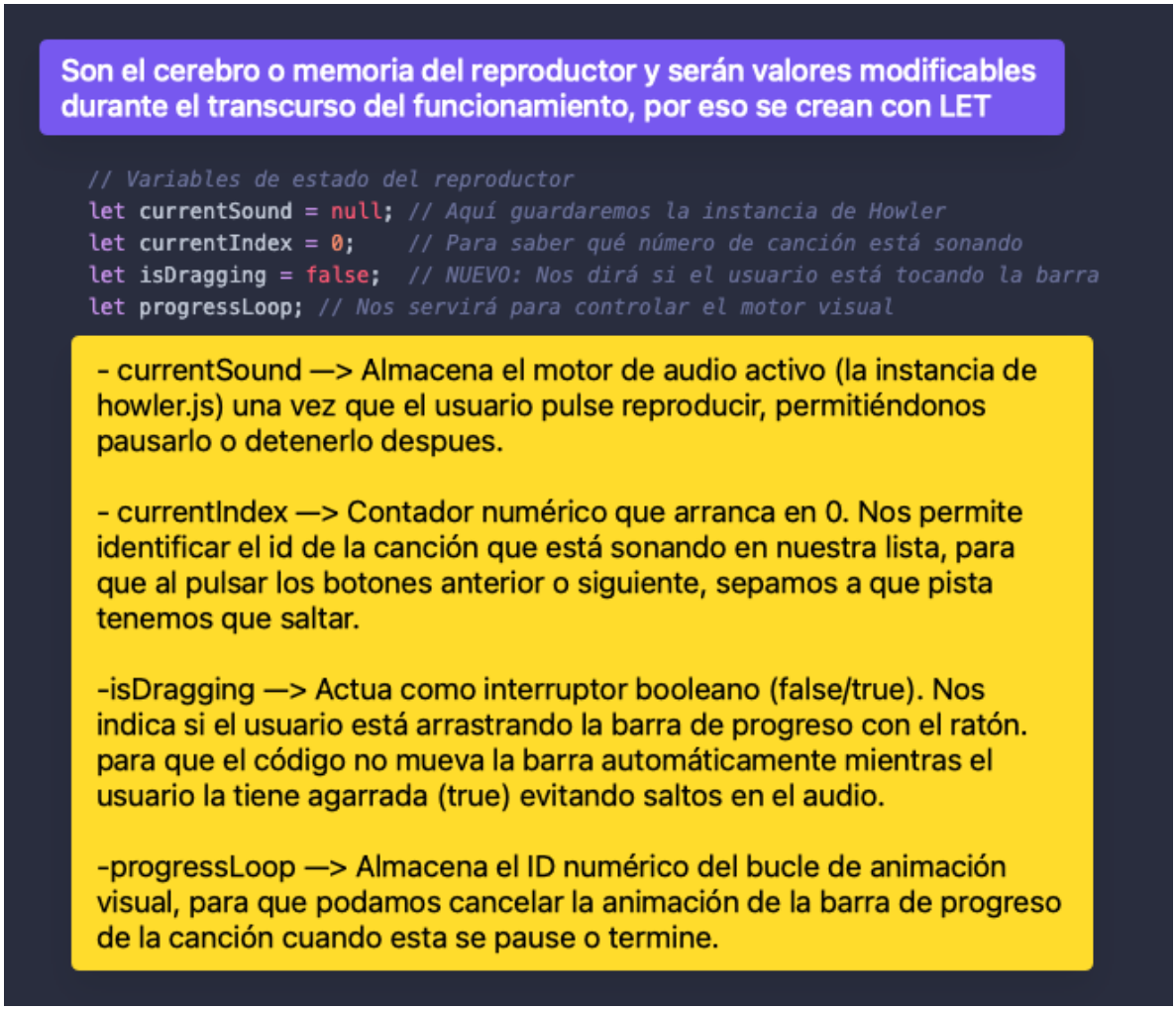
2) Creamos “puentes” hacia el HTML, denominadas REFERENCIAS AL DOM.



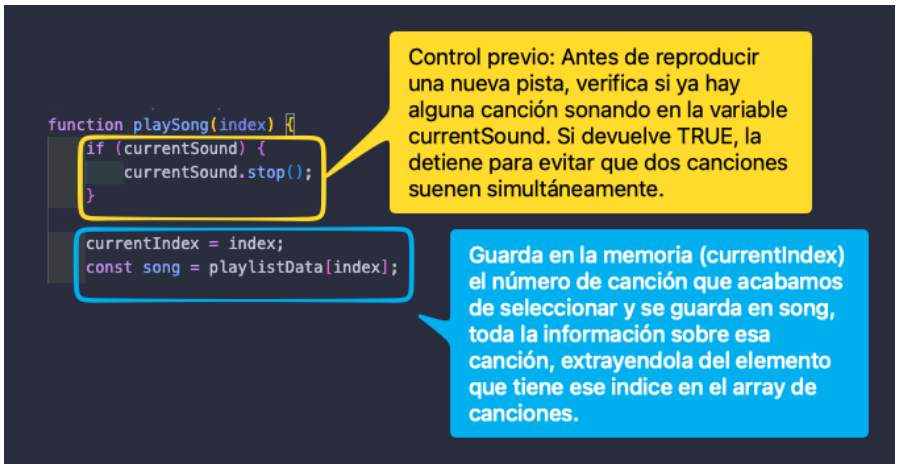
Repetimos esta tarea con cada uno de los elementos del DOM que necesitemos manipular dinámicamente con JavaScript.



3) Creamos variables de estado que serán necesarias para el funcionamiento de Howler.js



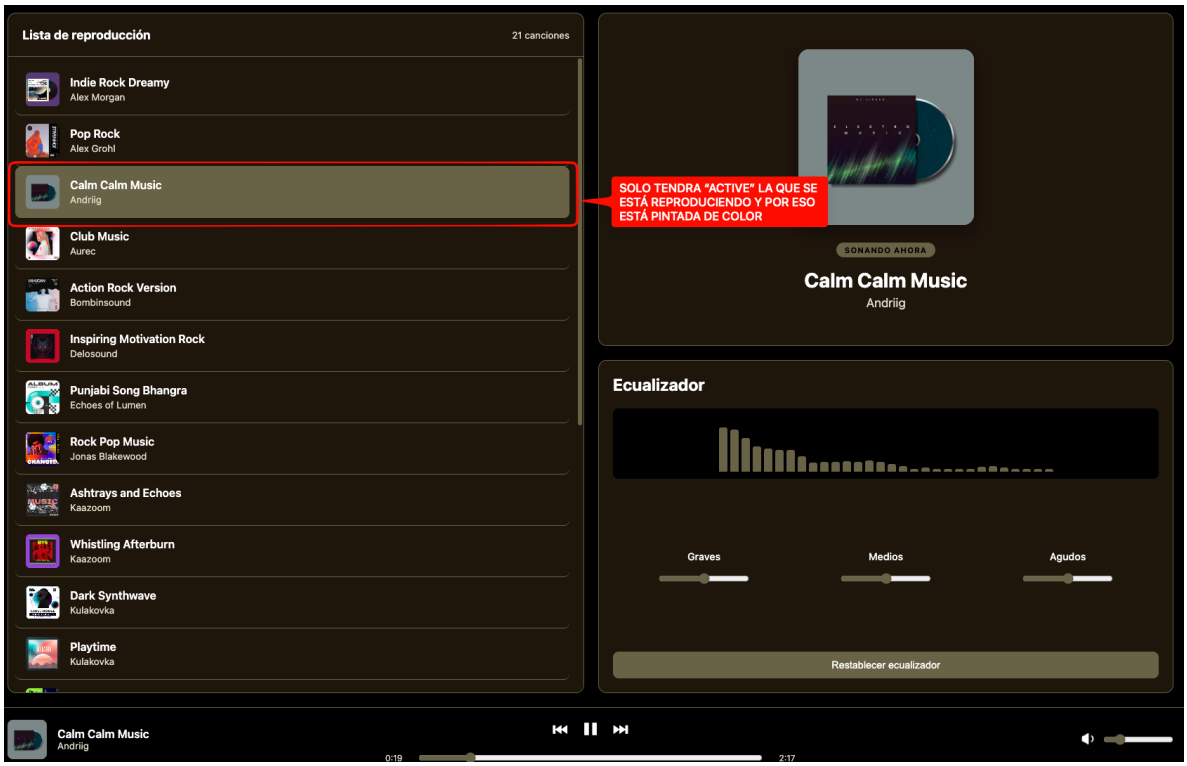
4) Creamos la función que actuará como el corazón absoluto de todo el reproductor musical. Se encarga de coordinar el cambio de canciones, configurar el motor de audio y actualizar todo lo que el usuario ve en la pantalla.



5) Realizamos la primer modificación visual: Marcar la canción que se está reproduciendo en la lista para que sea identificable



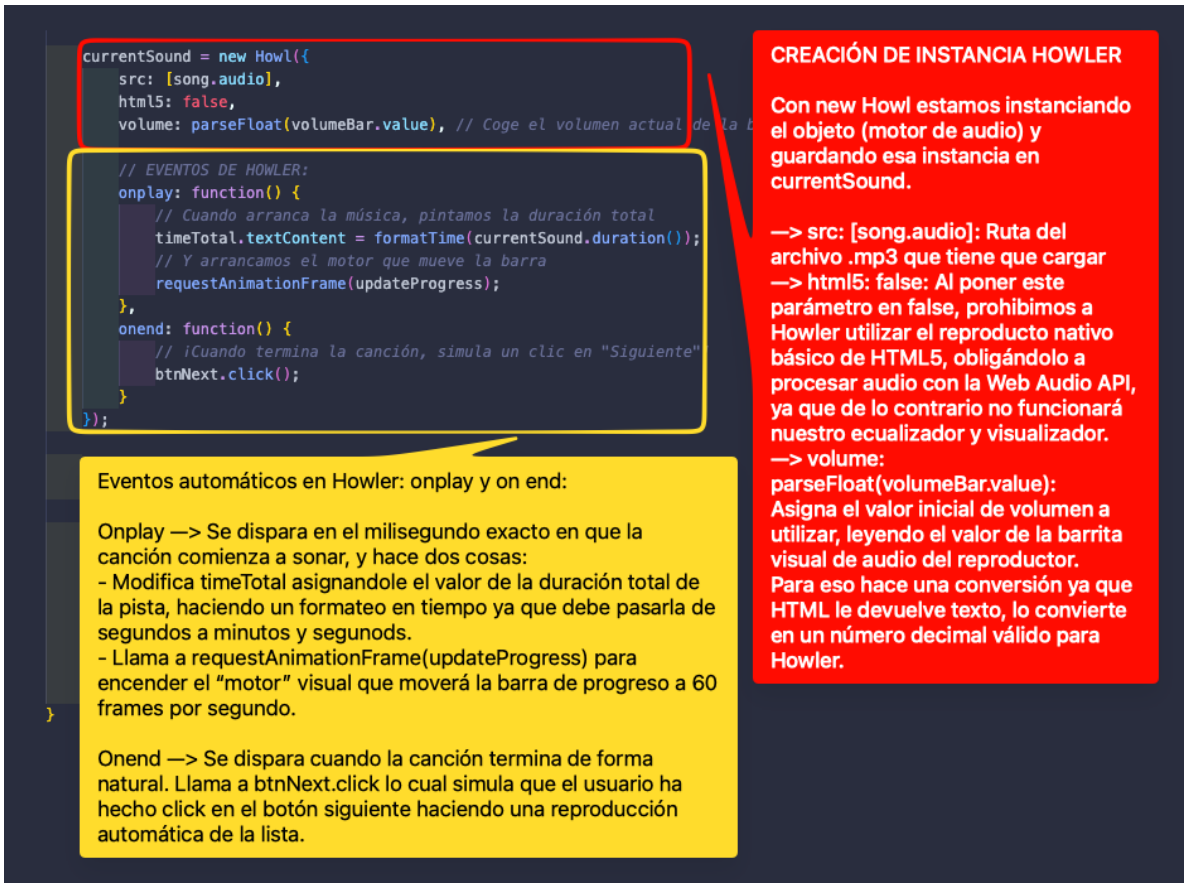
Con esto logramos una identificación visual de cual es la canción que esta sonando en la lista de reproducción:



NOTA: Las configuraciones del color para active se determinan en CSS

```css
/*11. ESTADO ACTIVO EN LA PLAYLIST*/
.playlist-item.active {
  background-color: var(--border-color);
  border-left: 4px solid var(--border-color);
  padding-left: 11px;
}
```

6) Configuramos el motor de audio (Howler.js)





7) Construcción visual de la lista de reproducción en pantalla

Hasta este momento la lista de reproducción muestra 0 canciones. Para cambiar esto y ver todas las canciones de la lista, crearemos una función llamada renderPlaylist(). Con esto logramos que en lugar de escribir a mano las 21 canciones en el archivo index.html se construyan dinámicamente leyendo nuestra BBDD (en este caso, el array que almacena los datos del .json).

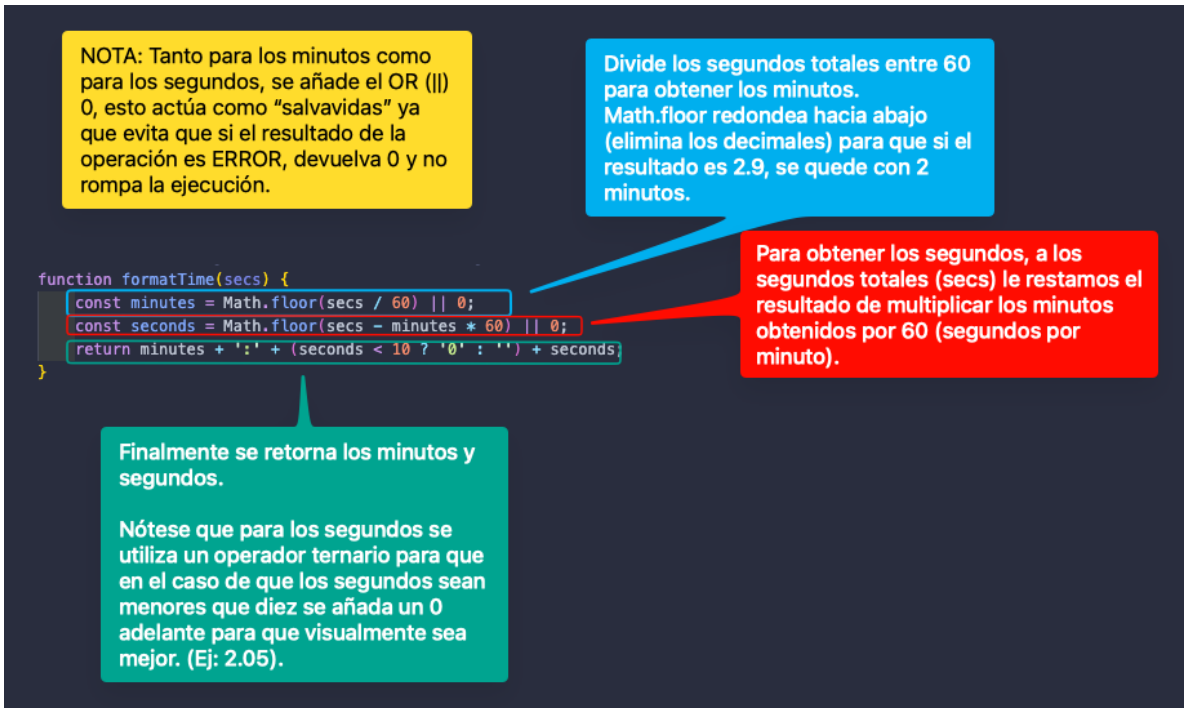


8) Creación y explicación de las funciones utilizadas anteriormente: formatTime() y updateProgress()

En el punto 6 hicimos la llamada de estas funciones para su utilización, ahora veremos que hace cada una y como se crean

- formatTime(secs)

  Las librerías de audio como Howler y los navegadores, siempre devuelven la duración en segundos totales (por ejemplo, 125.5 segundos). Esto no es visualmente tan agradable para las personas, por eso creamos esta función que "traduce" esos segundos totales en el formato clásico de minutos y segundos.



- updateProgress()

Esta función, es un "bucle infinito" que se ejecuta constantemente, calculando en tiempo real dónde debería estar nuestra barrita naranja



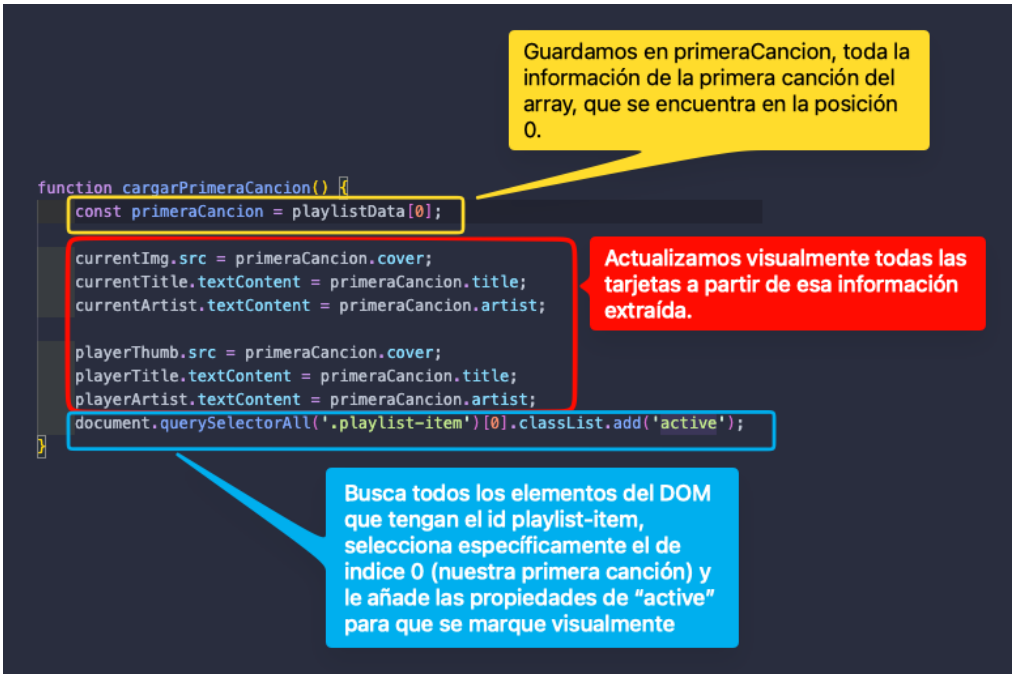


9) Inicializando la aplicación lista para usarse evitando datos genéricos

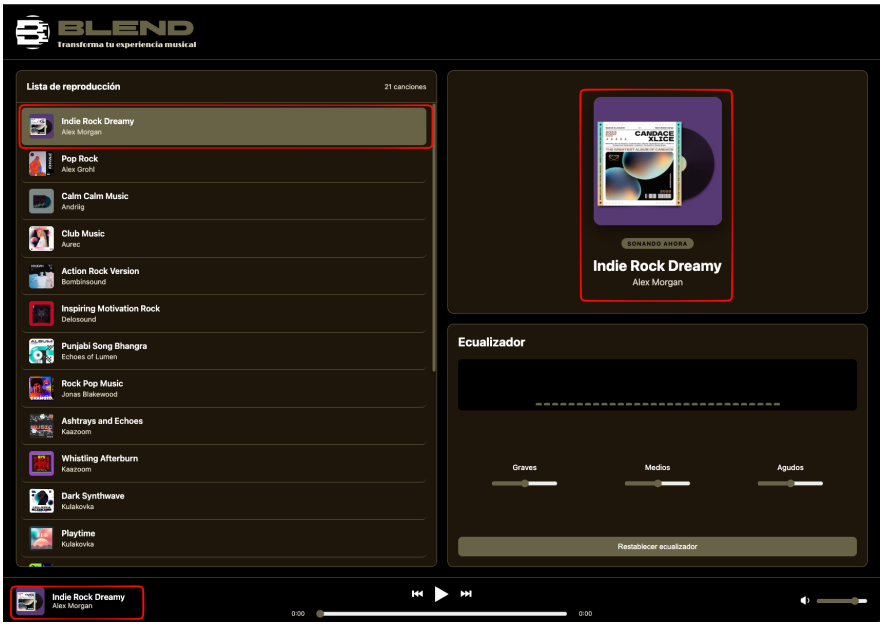
La siguiente función (cargarPrimeraCancion()) tiene un propósito puramente visual y de experiencia de usuario.

Se asegura que al entrar a la página web por primera vez, el reproductor no esté vacío ni con textos de relleno, sino que muestre los datos reales de la primera cancion de la lista, dejándola lista para reproducirse.

Como por defecto, si al entrar a la página principal pulsamos el botón play se reproduce la canción con id=0 (la primera), lo preparamos visualmente de esa manera.



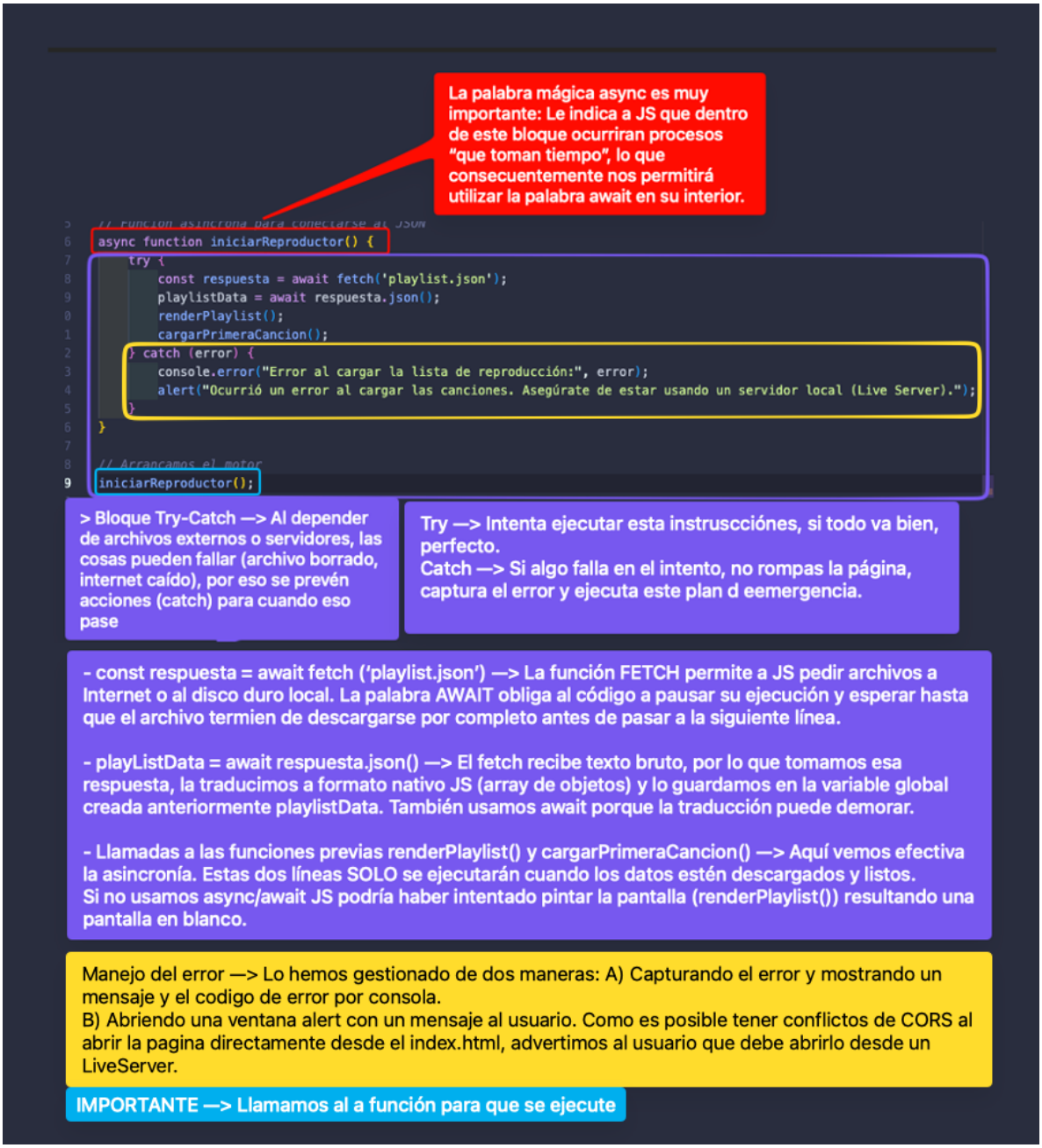
Como resultado, apenas cargar la página, ya tenemos información de lo que va a sonar al darle play:

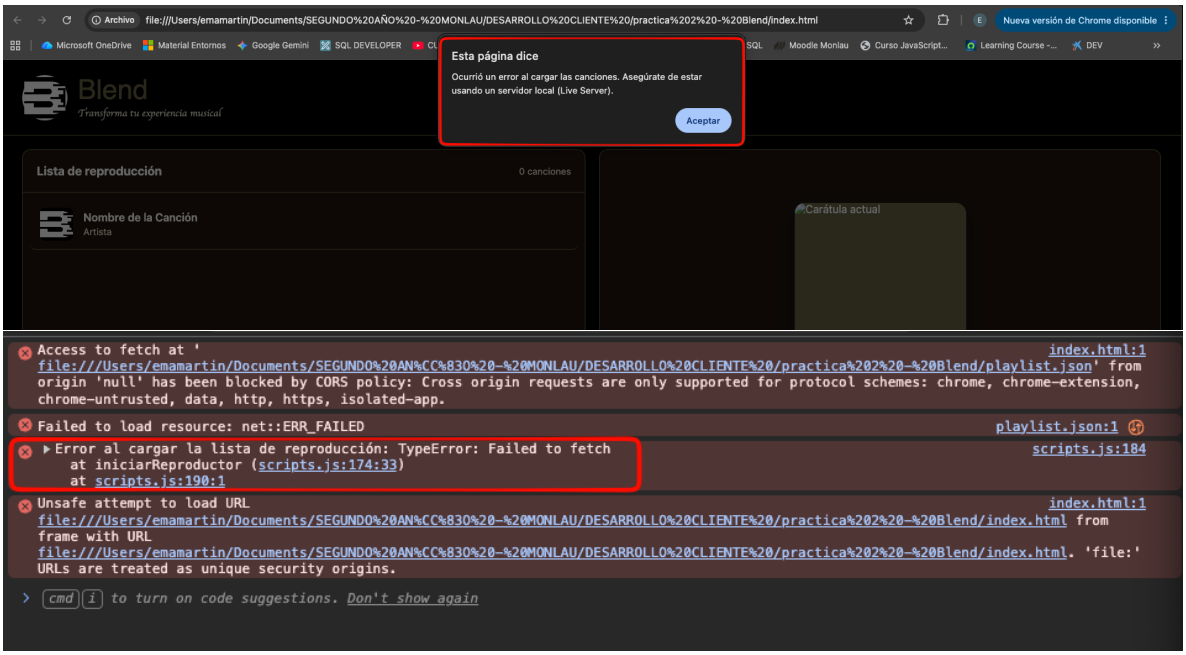


10) Creando el motor de arranque → iniciarReproductor()

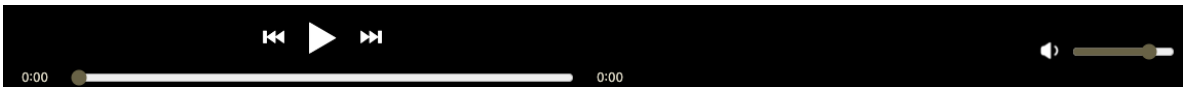
Esta función es la encargada de ir a buscar los datos de las canciones a nuestro archivo externo (.json) y asegurarse de que todo esté listo, ANTES de dibujar la interfaz.

Como estamos trabajando con un archivo externo .json, tenemos que empezar a trabajar con el asincronismo → Detiene la ejecución de la función hasta que se hayan recibido los datos correctamente.





11) Gestionando los eventos de los controles



Aquí comenzamos a trabajar los controles de reproducción prinicpales:

- Play/Pause
- Siguiente/anterior
- Barra de progreso
- Volumen

En este punto hacemos interactiva la página, convirtiendo los clicks del usuario en acciones reales, utilizando addEventListener (escuchadores de eventos).

- Play/Pause

```
// Evento para el botón de Play/Pause
btnPlayPause.addEventListener('click', () => {
    if (currentSound) {
        if (currentSound.playing()) {
            currentSound.pause(); // La pausamos
            btnPlayPause.textContent = '▶'; // Cambiamos el icono a Play
        }
        else {
            currentSound.play(); // La reanudamos
            btnPlayPause.textContent = '⏸'; // Cambiamos el icono a Pausa
        }
    } else {
        /*Si no hay ninguna canción cargada y le damos a play,
        reproducimos la primera de la lista por defecto.*/
        playSong(0);
    }
});
```

* btnPlayPause.addEventListener('click')... —> Se le añade un escuchador de eventos al elemento HTML almacenado en btnPlayPause de tipo click, donde a partir de este evento, se realiza lo siguiente:

1) if(currentSound) —> Corroboramos si efectivamente existe alguna pista cargada en la memoria. Si es TRUE, se desencadena la ejecución:

- if(currentSound.playing() —> Con .playing le preguntamos a Howler si la pista se está reproduciendo, si devuelve TRUE, la detenemos momentáneamente con currentSound.pause y modificamos el icono.

- Si currentSound.playing() arroja FALSE, ejecutamos currentSound.play() lo que permite reanudar y cambiamos el icono.

2) else —> Prevé que cuando no hay ninguna canción cargada en currentSound y se hace click en play, se ejecute instantáneamente la primera canción de la lista con playSong(0)

- Anterior/Siguiente

```
// Evento para el botón de Siguiente (⏭)
btnNext.addEventListener('click', () => {
    let nextIndex = currentIndex + 1;
    if (nextIndex >= playlistData.length) {
        nextIndex = 0;
    }
    playSong(nextIndex);
});

// Evento para el botón de Anterior (⏮)
btnPrev.addEventListener('click', () => {
    let prevIndex = currentIndex - 1;
    if (prevIndex < 0) {
        prevIndex = playlistData.length - 1;
    }
    playSong(prevIndex);
});
```

Se añaden dos escuchadores de eventos a los elementos HTML almacenados en las variables btnNext y btnPrev de tipo click, que dispararan los siguientes eventos:

- btnNext:
1) Calcula cual es debe ser la proxima canción a reproducir guardando en nextIndex el resultado de sumarle 1 a currentIndex (indice actual).
* A partir de allí evalúa: Si nextIndex es mayor o igual (recordemos que length arroja la cantidad de elementos, lo cual devuelve siempre uno mas que las posiciones) que el tamaño total de la lista, quiere decir que tiene que volver al Inicio porque no hay más canciones siguientes por reproducir, por lo que se autoasigna el valor 0.
Ejecuta playSong(nextIndex).

-btnPrev:
1) Calcula cual debe ser la canción anterior a reproducir guardando en prevIndex el resultado de restarle 1 a currentIndex (indice actual).
* A partir de allí evalúa: Si prevIndex es menor que 0, quiere decir que estaba reproduciendo la primer canción de la lista (indice 0), por lo que la canción anterior, será la última de la lista. Para ello se autoasigna el resultado de restarle a la lista total de elementos 1, teniendo en cuenta lo que dijimos respecto a que la cantidad de leementos siempre es uno mas que su posición.

- Barra de progreso

En este caso trabajaremos con eventos diferentes a click, utilizando “mousedown” y “touchstart”

Se han utilizado los dos para preveer la posibilidad de que la ejecución se realice tanto con un ratón normal como con una pantalla táctil.



- Control de volumen



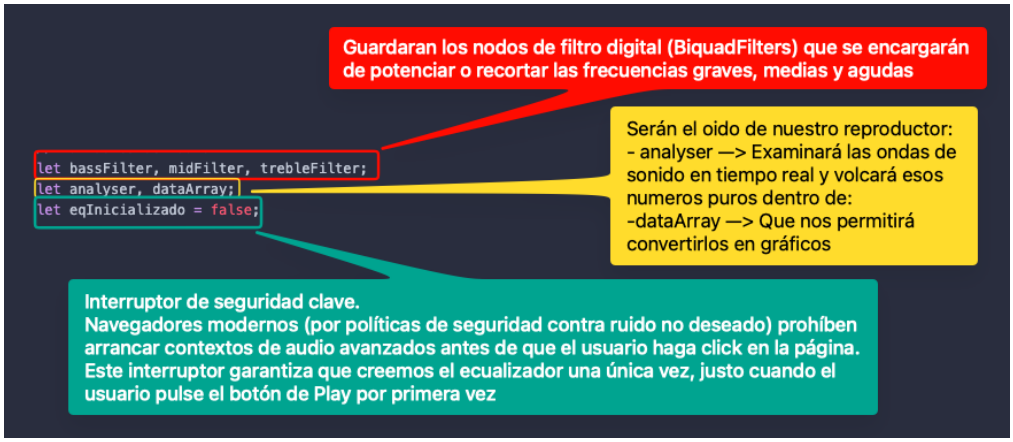
12) ECUALIZADOR Y VISUALIZADOR

Para trabajar con la sección del ecualizador, también tendremos que capturar los elementos del DOM en variables JS correspondientes para poder trbajarlos.



<u>Crearemos variables de audio y variables visuales</u>

- Variables de audio → Se crean vacias (con let) y actuarán como contenedores para las piezas de nuestro equipo de sonido digital:



➢ Variables visuales

```
const NUM_BARS = 30; // Número de barras que tendrá el visualizador
const visualizerBars = []; // Aquí guardaremos las barras HTML
```

- const NUM_BARS =30 —> Definimos como constante inmutable que queremos exactamente 30 barras bailando al ritmo de la música

- const visualizerBars=[] —> Creamos una lista vacía, que almacenará una referencia directa a cada una de esas 30 barras de HTML que vamos a generar.

Guardarlas en esta memoria es clave para el rendimiento: Nos permitirá animarlas a 60 frames por segundo sin obligar al navegador a buscar en el HTML constantemente.

- Generación de las barras

En lugar de escribir 30 etiquetas a mano en el archivo HTML, las construimos dinámicamente con un bucle for, que se ejecuta exactamente 30 veces (el valor que le asignamos a NUM_BARS)

```
// GENERAR LAS BARRAS HTML UNA SOLA VEZ
for(let i = 0; i < NUM_BARS; i++) {
    const bar = document.createElement('div');
    bar.className = 'bar';
    visualizerContainer.appendChild(bar);
    visualizerBars.push(bar); // Guardamos la referencia para animarlas luego
}
```

En cada una de esas 30 vueltas, se crea un elemento div (document.createElement('div'), se le asigna una clase CSS 'bar' y se lo inyecta dentro del contenedor (visualizerContainer.appendChild(bar).

Luego se guarda una referencia de esa nueva barra en el array visualizerBars que creamos vacío, usando el metodo push(). Esto nos permitirá posteriormente animarlas sin tener que buscar cada barra en el HTML en cada fotograma.

- La función iniciarEqualizador()

```
// INICIAR EL ECUALIZADOR Y EL ANALIZADOR
function iniciarEcualizador() {
    if (eqInicializado || !Howler.ctx) return;

    // Creamos los Filtros del Ecualizador
    bassFilter = Howler.ctx.createBiquadFilter();
    bassFilter.type = 'lowshelf';
    bassFilter.frequency.value = 200;

    midFilter = Howler.ctx.createBiquadFilter();
    midFilter.type = 'peaking';
    midFilter.frequency.value = 1000;
    midFilter.Q.value = 1;

    trebleFilter = Howler.ctx.createBiquadFilter();
    trebleFilter.type = 'highshelf';
    trebleFilter.frequency.value = 3000;

    analyser = Howler.ctx.createAnalyser();
    analyser.fftSize = 128; // Resolución del análisis (nos dará 64 barras de datos)
    const bufferLength = analyser.frequencyBinCount;
    dataArray = new Uint8Array(bufferLength);
    Howler.masterGain.disconnect();

    Howler.masterGain.connect(bassFilter);
    bassFilter.connect(midFilter);
    midFilter.connect(trebleFilter);
    trebleFilter.connect(analyser);
    analyser.connect(Howler.ctx.destination);

    eqInicializado = true;

    // Arrancamos el motor de la animación visual
    requestAnimationFrame(animateVisualizer);
}
```

Antes de introducirnos en el código del ecualizador, haremos una breve explicación de qué es el sonido para poder entender mejor el funcionamiento.

El sonido son vibraciones:

- Si algo vibra muy lento, hace un sonido GRAVE.
- Si algo vibra muy rápido, hace un sonido AGUDO.

En el mundo del audio, esta velocidad de vibración se mide en Hercios (Hz), por lo que números bajos (por ejemplo 200) son graves, y números altos (por ejemplo 3000) son agudos.

En el ecualizador que hemos trabajado, hemos creado tres botones o ruedas (llamadas BiquadFilter). Estos botones en lugar de subir el volúmen de toda la canción, solo suben o bajan el volúmen de un tipo de sonido.

- El filtro de Graves (en el código bassFilter) lo configuramos en 200 Hz (Vibraciones lentas).
  Este botón controla el bajo, bombos de batería y los “golpes” de la música.

- El filtro de Medios (en el código midFilter) lo configuramos a 1000 Hz. Aquí vive la voz humana, las guitarras y los pianos. Si subimos este filtro, por ejemplo, la voz humana se escucha más cercana.
- El filtro de Agudos (trebleFilter) lo configuramos a 3000 Hz (Vibraciones rápidas). Esto controla platillos del a atería, o el sonido de la letra “S” al cantar por ejemplo.



Creamos el “escuchador” constante de la música:

El AnalyserNode actúa como un micrófono que escucha dentro de nuestro reproductor de música.

No altera el sonido, sino que escucha la canción a una velocidad de 60 veces por segundo, pasándonos un reporte numérico.

Por ejemplo → En X milisegundo de la canción, la música está sonando con una fuerza de 80 sobre 255. Estos números nos permiten decirle a las barras HTML del ecualizador, el tamaño hacia arriba o hacia abajo que tienen que tomar, creando la animación visual.

```
analyser = Howler.ctx.createAnalyser();
analyser.fftSize = 128; // Resolución del análisis (nos dará 64 barras de datos)
const bufferLength = analyser.frequencyBinCount;
dataArray = new Uint8Array(bufferLength);
Howler.masterGain.disconnect();
```

La conexión de todos los elementos:

Teniendo los tres botones de control de sonidos y el “espía” analyser, debemos crear una conexión entre ellos.

Para que todo funcione la música tiene que viajar en un orden estricto.

Para ello creamos la orden .connect() para “enchufar” cada cable de un sitio a otro, creando este camino:

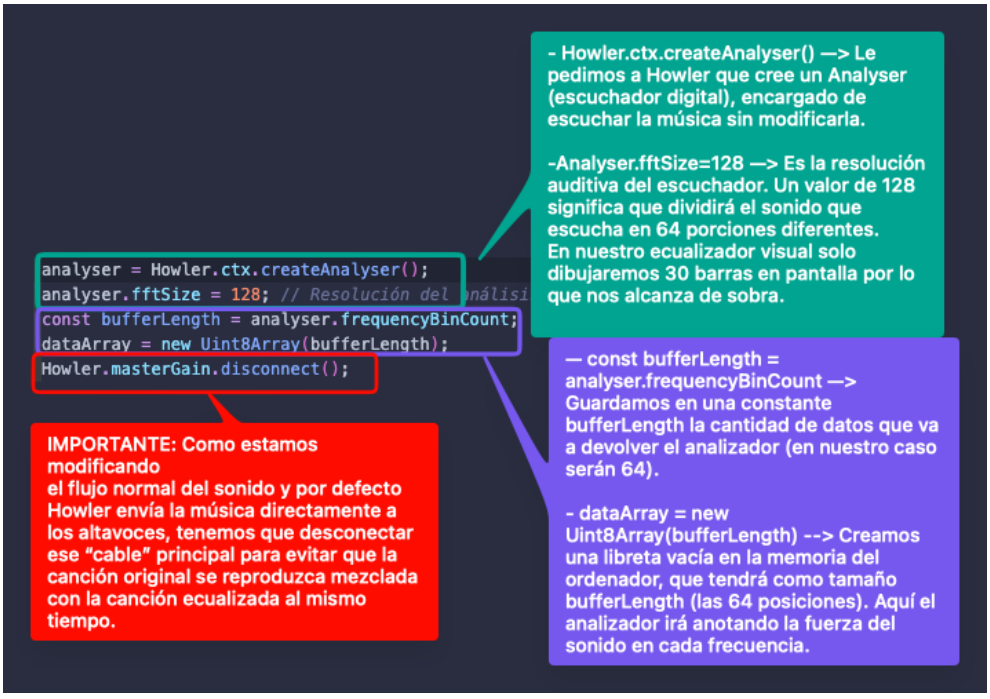
1) Origen: La canción pura sale del reproductor Howler
2) Cable 1: Pasa por el botón de Graves (Para ver si hemos subido o bajado)
3) Cable 2: Pasa por el botón de Medios
4) Cable 3: Pasa por el botón de Agudos
5) Cable 4: Con el sonido neutral o modificado por nuestros cables anteriores, el sonido es escuchado por el espía (Analyser) para que haga su reporte sobre la fuerza del sonido y nos permita luego general las barras visualez.
6) El destino final: Analyser envía la señal a los altavoces del ordenador (destination) para que pueda ser escuchada.

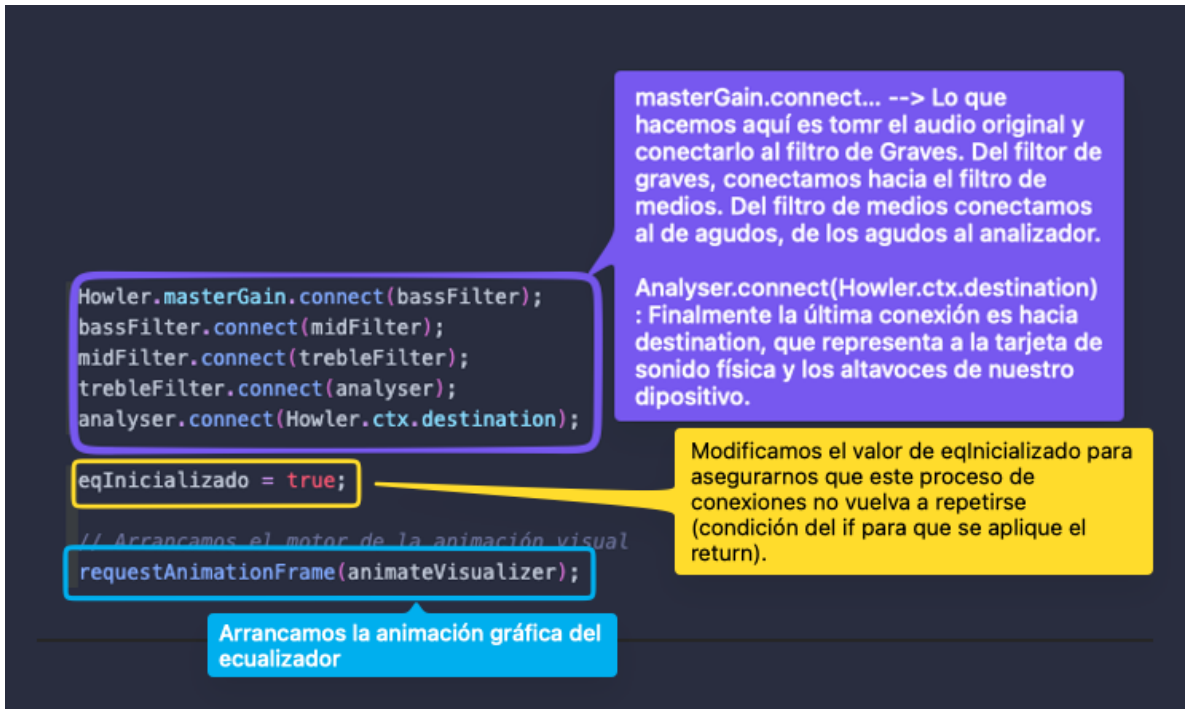
```
Howler.masterGain.connect(bassFilter);
bassFilter.connect(midFilter);
midFilter.connect(trebleFilter);
trebleFilter.connect(analyser);
analyser.connect(Howler.ctx.destination);

eqInicializado = true;
```

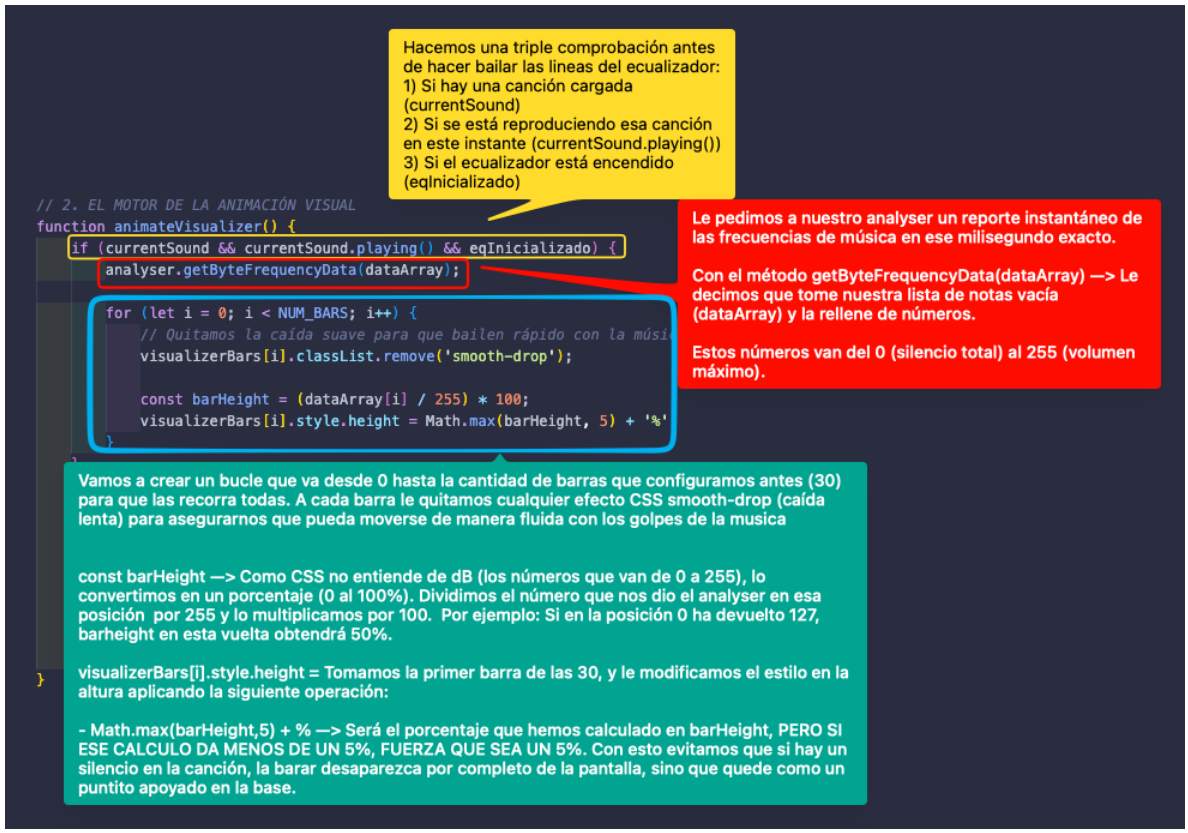
Teniendo en cuenta la explicación anterior, ahora sí desglosaremos el código:


// INICIAR EL ECUALIZADOR Y EL ANALIZADOR
function iniciarEcualizador() {
if (eqInicializado || !Howler.ctx) return;
bassFilter = Howler.ctx.createBiquadFilter();
bassFilter.type = 'lowshelf';
bassFilter.frequency.value = 200;
midFilter = Howler.ctx.createBiquadFilter();
midFilter.type = 'peaking';
midFilter.frequency.value = 1000;
midFilter.Q.value = 1;
trebleFilter = Howler.ctx.createBiquadFilter();
trebleFilter.type = 'highshelf';
trebleFilter.frequency.value = 3000;
Puerta de seguridad.
Verificamos si eqInicializado=true, quiere decir que el ecualizador ya ha sido encendido antes y retorna la ejecución para evitar enchufar los cables dos veces ya que distorsionaria el audio
Verificamos si !Howler.ctx = true quiere decir que el reproductor de audio de Howler aún no existe, por lo que retorna para evitar que el código de error por intentar modificar un audio que aún no ha sido cargado
createBiquadFilter() —> Da la orden al motor de audio de Howler de fabricarnos un filtro digital
Aquí variaran los .type:
- Lowshelf —> Afectará todas las frecuencias que estén por debajo del límite que le marquemos.
Definimos el limite de la frecuencia en 200
- Peaking —> Solo afecta las frecuencias que estén justo alrededor del valor que le demos.
Definimos esa frecuencia en 1000.
Con Q.value definimos que tan ancha es esa campana.
- Highself —> Lo contrario a los graves, afectará todas las frecuencias que estén por encima del límite que le marquemos.
Definimos el limite de la frecuencia en 3000.

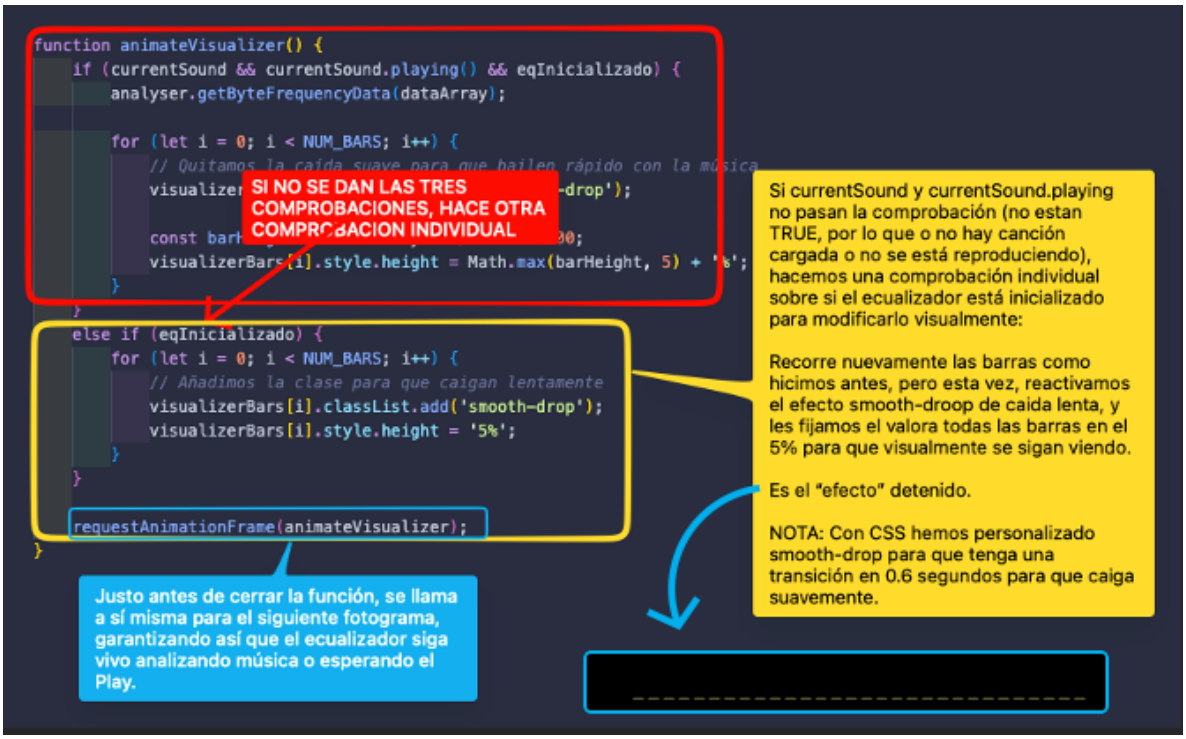

analyser = Howler.ctx.createAnalyser();
analyser.fftSize = 128; // Resolución del análisi
const bufferLength = analyser.frequencyBinCount;
dataArray = new Uint8Array(bufferLength);
Howler.masterGain.disconnect();
- Howler.ctx.createAnalyser() —> Le pedimos a Howler que cree un Analyser (escuchador digital), encargado de escuchar la música sin modificarla.
-Analyser.fftSize=128 —> Es la resolución auditiva del escuchador. Un valor de 128 significa que dividirá el sonido que escucha en 64 porciones diferentes.
En nuestro ecualizador visual solo dibujaremos 30 barras en pantalla por lo que nos alcanza de sobra.
— const bufferLength = analyser.frequencyBinCount —> Guardamos en una constante bufferLength la cantidad de datos que va a devolver el analizador (en nuestro caso serán 64).
- dataArray = new Uint8Array(bufferLength) --> Creamos una libreta vacía en la memoria del ordenador, que tendrá como tamaño bufferLength (las 64 posiciones). Aquí el analizador irá anotando la fuerza del sonido en cada frecuencia.
IMPORTANTE: Como estamos modificando
el flujo normal del sonido y por defecto Howler envía la música directamente a los altavoces, tenemos que desconectar ese "cable" principal para evitar que la canción original se reproduzca mezclada con la canción ecualizada al mismo tiempo.



Animación gráfica del ecualizador:




function animateVisualizer() {
if (currentSound && currentSound.playing() && eqInicializado) {
analyser.getByteFrequencyData(dataArray);
for (let i = 0; i < NUM_BARS; i++) {
// Quitamos la caída suave para que bailen rápido con la música
visualizer -drop');
const barH 0;
visualizerBars[i].style.height = Math.max(barHeight, 5) + '%';
}
}
else if (eqInicializado) {
for (let i = 0; i < NUM_BARS; i++) {
// Añadimos la clase para que caigan lentamente
visualizerBars[i].classList.add('smooth-drop');
visualizerBars[i].style.height = '5%';
}
}
requestAnimationFrame(animateVisualizer);
}
SI NO SE DAN LAS TRES COMPROBACIONES, HACE OTRA COMPROBACION INDIVIDUAL
Si currentSound y currentSound.playing no pasan la comprobación (no estan TRUE, por lo que o no hay canción cargada o no se está reproduciendo), hacemos una comprobación individual sobre si el ecualizador está inicializado para modificarlo visualmente:
Recorre nuevamente las barras como hicimos antes, pero esta vez, reactivamos el efecto smooth-droop de caida lenta, y les fijamos el valora todas las barras en el 5% para que visualmente se sigan viendo.
Es el "efecto" detenido.
NOTA: Con CSS hemos personalizado smooth-drop para que tenga una transición en 0.6 segundos para que caiga suavemente.
Justo antes de cerrar la función, se llama a sí misma para el siguiente fotograma, garantizando así que el ecualizador siga vivo analizando música o esperando el Play.


function animateVisualizer() {
if (currentSound && currentSound.playing() && eqInicializado) {
analyser.getByteFrequencyData(dataArray);
for (let i = 0; i < NUM_BARS; i++) {
// Quitamos la caída suave para que bailen rápido con la música
visualizer -drop');
const barH 0;
visualizerBars[i].style.height = Math.max(barHeight, 5) + '%';
}
}
else if (eqInicializado) {
for (let i = 0; i < NUM_BARS; i++) {
// Añadimos la clase para que caigan lentamente
visualizerBars[i].classList.add('smooth-drop');
visualizerBars[i].style.height = '5%';
}
}
requestAnimationFrame(animateVisualizer);
}
SI NO SE DAN LAS TRES COMPROBACIONES, HACE OTRA COMPROBACION INDIVIDUAL
Si currentSound y currentSound.playing no pasan la comprobación (no estan TRUE, por lo que o no hay canción cargada o no se está reproduciendo), hacemos una comprobación individual sobre si el ecualizador está inicializado para modificarlo visualmente:
Recorre nuevamente las barras como hicimos antes, pero esta vez, reactivamos el efecto smooth-droop de caida lenta, y les fijamos el valora todas las barras en el 5% para que visualmente se sigan viendo.
Es el "efecto" detenido.
NOTA: Con CSS hemos personalizado smooth-drop para que tenga una transición en 0.6 segundos para que caiga suavemente.
Justo antes de cerrar la función, se llama a sí misma para el siguiente fotograma, garantizando así que el ecualizador siga vivo analizando música o esperando el Play.

Configurando los cambios en la ecualización por el usuario

En este punto recibiremos los movimientos que se hagan en los tres botones del ecualizador por parte del usuario, para realizar los cambios en el sonido al reproducirse la canción.

```
// 3. EVENTOS DE LAS BARRAS DEL ECUALIZADOR
eqBass.addEventListener('input', (e) => {
    if (eqInicializado) bassFilter.gain.value = parseFloat(e.target.value);
});
eqMid.addEventListener('input', (e) => {
    if (eqInicializado) midFilter.gain.value = parseFloat(e.target.value);
});
eqTreble.addEventListener('input', (e) => {
    if (eqInicializado) trebleFilter.gain.value = parseFloat(e.target.value);
});
```

Se crean 3 eventos, uno para cada una de las barras de control del tipo de sonido.

Los 3 eventos hacen lo mismo. Programamos eventos de tipo INPUT que alterarán el sonido en tiempo real.

Usamos INPUT (a diferencia de change que utilizamos en la barra de reproducción la cual solo se activa al soltar el click del ratón) porque se dispara decenas de veces por segundos mientras el usuario desliza la barra, lo cual le permite escuchar como van cambiando los sonidos al instante.

—> Barrera de seguridad: Solamente ejecutamos el cambio si eqInicializado da TRUE. De lo contrario, si no hay ecualizador iniciado y el usuario mueve las barras, el codigo intentaria cambiar el sonido de un filtro que no existe y rompería la página.

Añadimos a cada filtro en particular la propiedad .gain.value que es la rueda de volumen de cada filtro especifico y parseamos el valor que nos da entrega la barra HTML (que antes limitamos de 40 a -40) ya que nos lo entrega en texto plano.

Previsión de un botón restablecer

Como le estamos permitiendo al usuario realizar modificaciones, puede que necesite volver a los valores iniciales.

Por eso establecemos unas líneas de códigos sencilla que nos permiten devolver los valores iniciales a cada uno de los filtros:

```
// 4. BOTÓN DE RESTABLECER
btnResetEq.addEventListener('click', () => {
    eqBass.value = 0; eqMid.value = 0; eqTreble.value = 0;
    if (eqInicializado) {
        bassFilter.gain.value = 0;
        midFilter.gain.value = 0;
        trebleFilter.gain.value = 0;
    }
});
```

Le asignamos un escuchador de eventos de tipo click al boton reset.

Aquí hacemos dos cosas:

1) La modificación VISUAL —> Las tres barras volverán a ubicarse en el centro (el valor 0).

2) La modificación SONORA —> Los filtros del ecualizador vuelven a valores 0 y se reestablece el audio original.

